

KNN-Q Learning Algorithm of Bitrate Adaptation for Video Streaming over HTTP

HuaiDi Lin, ZhenYuan Shen, HuaKang Zhou, XingGuang Liu, Leilei Zhang, Gang Xiao, Zhenbo Cheng

Dept. of Computer Science and Technology

Zhejiang University of Technology

Hangzhou, China

e-mail: {xg, czb}@zjut.edu.cn

Abstract—The increasing demands for seamless switching of high definition video streams have become the main driving force for the research of bitrate adaptive algorithms. Existing Q-learning based adaptive bitrate algorithm can obtain higher quality of experience (QoE) through dynamically learning the optimal behavior corresponding to the current network environment. However, ordinary Q-learning algorithm has the problem that it is difficult to encode continuous state values and the algorithm converges slowly in large state spaces. In this article, we combine the nearest neighbor (KNN) with Q-learning algorithms to propose a KNN-Q learning algorithm that can deal with continuous state. Our experimental results show that in the bitrate adaptive scenario, the KNN-Q learning can achieve higher QoE and faster convergence speed than Q-learning algorithm.

Keywords—DASH; HTTP adaptive streaming; reinforcement learning; Q-learning; K-Nearest Neighbor

I. INTRODUCTION

In recent years, applications based on video data are becoming the mainstream application scenarios of traditional Internet and mobile Internet. According to the white paper of Cisco Global Mobile Data Forecast (2016-2021), mobile video is expected to grow 8.7 times by 2021, and its traffic will account for 78% of total network mobile traffic[1]. Compared with other formats of data, such as text and pictures, the transmission of video data requires a higher bitrate, so the increase in network data traffic is more obvious. Users demand higher video quality and interactive features.

The MPEG organization formulated the dynamic adaptive streaming over HTTP (DASH) standard in 2011, which aims to unify video transmission technology standards for transmitting dynamic bitrates between different device and server manufacturers [2]. According to the DASH standard, the server divides the original video into video segments with the same content but different bitrates, and then generates a media presentation description (MPD) index file containing all video segment information and their download address. After the client downloads and parses the MDP file, it switches the video clip download at the appropriate bitrate during playback based on the current video quality and network conditions [3]. The DASH standard inspires the design of the bitrate adaptive algorithm so that developers can implement their own bitrate adaptive algorithms to provide higher QoE.

The existing bitrate adaptive algorithms are mostly based on heuristic algorithms [4]. The heuristic algorithms usually use hard coding to select the appropriate bitrate according to the network conditions, which may result in the inability to effectively cope with the frequent fluctuations of the network environment [5]. In addition, base station buffering is considered as a class of effective video streaming methods [6]. The cache time during off-peak hours can make popular content closer to users, thereby improving users' QoE [7]. However, the base station buffering method does not consider the effect of bitrate allocation on the buffering effectiveness. Recently, many bitrate adaptive algorithms based on reinforcement learning algorithms have been proposed to solve the problem of the impact of bitrate allocation on the buffer [5, 8-11]. However, traditional reinforcement learning-based algorithms, such as Q-learning, often assume that the bitrate is discrete. The purpose of setting the bitrate to discrete values is to simplify real-world modeling. This simplification will lead to poor test performance. Another type of reinforcement learning algorithm uses deep neural networks to simulate bitrate values [11, 12]. Although this can simulate the network environment more realistically, it needs a large amount of data to accurately simulate the network environment. For these reasons, this paper proposes that KNN can be used when the state space of bitrate is continuous. The simulation results show that the proposed algorithm can achieve higher QoE and faster convergence speed than Q-learning algorithm.

The rest of this paper is organized as follow. In Section II, we model the problem according to a Markov Decision Process. We propose the KNN-Q learning algorithm to address the bitrate adaptive problem in a dynamical network environment in Section III. Numerical results are presented in Section IV, and a conclusion is reached in Section V.

II. MODEL PROBLEM FORMULATION

The server encodes the original high-definition video at different bitrates, and each bit rate includes N video segments (Fig. 1). The DASH client determines from which bitrate to download a video segment according to network conditions, such as network bandwidth, buffer, etc., The downloaded video segment is represented as s_t , where $t = 1, 2, \dots, T$ represents the number of each video segment. Therefore, the selection of video segments in DASH client can be modeled as a decision-making process. It may be assumed that there is an agent in the DASH client. The

information that the agent can observe at number t includes network bandwidth, buffer, and video quality at number $t-1$. The agent needs to select which bitrate video segment to be download. The agent's goal is to allow the video to be played on the client at a high bitrate and smoothly when the network bandwidth changes dynamically.

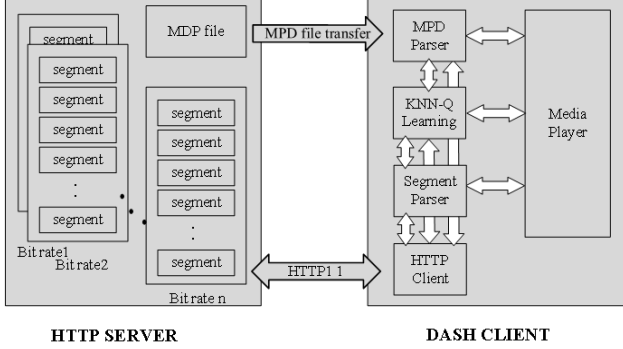


Figure 1. Communication in video streaming system

The client agent's decision process can be modeled using a Markov decision process (MDP). The MDP includes four-tuples, namely state space S , action space A , state transition probability P , and instant reward R . The state space depicts the environment of network. The action space is a set of selectable bitrates for all candidate video segments on the server. The policy of agent is the mapping from state to action, that is, how to choose a specific action at in a given state s_t . After the agent performs the action a_t , the state s_t will change into s_{t+1} accordingly. At this time, the state s_{t+1} perceived by the agent meets conditional probability $P(s_{t+1} | s_t, a_t)$. After performing an action, the agent evaluates the result of the action through instant reward R_t .

A state includes network bandwidth size BD_t , buffer B_t , and the quality of the previous video segment q_{t-1} . The quality of video q_t is expressed as the Structural Similarity Index (SSIM) [13], which has been chosen for its fidelity of the metric with the subjective quality. Typical values of SSIM range from 0.8 to 1, and below 0.8 represent poor quality video segments. The SSIM can be approximated as a fourth-degree polynomial of the logarithm of the normalized rate as follows:

$$q = SSIM \approx 1 + d_1(\rho_a) + d_2(\rho_a)^2 + d_3(\rho_a)^3 + d_4(\rho_a)^4 \quad (1)$$

where $\rho_a = \log(R_a / R_1)$, with R_1 being the maximum experienced bitrate and R_a being the encoding rate compressed by the encoding module on the video server. The vector $\mathbf{d} = [d_1, d_2, d_3, d_4]$ is a description of the complexity of a video.

We define the time required to download the video segment as

$$T_{download} = \frac{BR_t \times T_{segment}}{BD_t} \quad (2)$$

where $T_{segment}$ is (usually 2 seconds) the time length of the video segment. After the agent requests to download the video segment at time t , the buffer size B_{t+1} in the number $t+1$ will be updated based on the previous network bandwidth size BD_t and video bitrate BR_t , according to action a_t . The buffer size B_{t+1} can be updated as follows:

$$B_{t+1} = \max\{0, B_t - T_{download}\} + T_{segment} \quad (3)$$

We design the instant reward in this study to include three indicators that affect the QoE. The first indicator is the quality q_t of the video segment at t . The higher the quality of the video, the higher the QoE it obtains. The second indicator is the change of quality between two adjacent segments. The smaller the second indicator, the higher the QoE. The third indicator is the danger of data overflow in the buffer. The smaller the third indicator, the higher the QoE it obtains. Therefore, the reward function is

$$R(t) = q_t - \alpha|q_t - q_{t-1}| - \varphi(t) \quad (4)$$

where $\varphi(t)$ is a penalty term for measuring the danger of data overflow in the buffer.

The setting of this penalty item first needs to ensure that the client is at a safe buffer level and avoid actions that may cause low buffer. The low buffer action will trigger a re-buffer event, that is, before the next number of video segment starts to be downloaded, the re-buffer event will be triggered when the current buffer size is 0. Re-buffer event can greatly reduce QoE, so the agent should try to avoid this event in the decision-making process.

The third penalty item $\varphi(t)$ in (4) is got according to

$$\varphi(t) = \min\{\alpha(\max[0, T_{download} - B_t]), 1\} + \beta(\max[B_M - B_{t+1}, 0])^2 \quad (5)$$

where α and β determines the evaluation tendency, and they are used to adjust whether the evaluation pays attention to the level of secure buffer and the occurrence of re-buffer events. B_M is a safe buffer level.

The goal of agent is to obtain the best long-term rewards according to a policy. The long-term rewards function is defined as follows:

$$V(s_0) = E[\sum_{t=0}^T \lambda^t R_t] \quad (6)$$

where E means take expectations, and s_0 is the initial state, and λ is a discount factor between 0 and 1.

III. SOLVE DECISION PROCESS PROBLEM

In order to find a policy that satisfies (6), a commonly used reinforcement learning algorithm is Q-learning. In Q-learning, the Q function is used to measure the long-term returns of choosing action a in state s , and the agent continuously updates the Q function in interaction with the environment to obtain an estimate of the long-term returns after taking action a in state s . The learning process is that the estimated value gradually approaches the true long-term

return value. It can be proved that the algorithm can converge when the state transition meets certain conditions, that is, the estimated value of the long-term return after the action a in the state s is equal to the true value [14]. The updated equation of the Q function is as follows:

$$Q(s_t, a_t) = Q(s_t, a_t) + \eta[r_{t+1} + \gamma \max_b Q(s_{t+1}, b) - Q(s_t, a_t)] \quad (7)$$

where η and γ are the learning rates between 0 and 1.

According to the updated formula of the Q function in (7), it can be known that when the agent takes an action a_t at state s_t , the state will be converted to s_{t+1} , and the agent obtain a reward r_{t+1} . At this time, the action b corresponding to state s_{t+1} needs to be selected to take the maximum value $Q(s_{t+1}, b)$, and $Q(s_{t+1}, b)$ is used to update $Q(s_t, a_t)$.

When the agent is in a state, a specific action can be selected according to the Q function value corresponding to each action. The probability that the action a_t is selected at state s_t is given as

$$P(a_t) = \frac{e^{\frac{Q(s_t, a_t)}{\tau}}}{\sum_{i=1}^{\text{number of actions}} e^{\frac{Q(s_t, a_i)}{\tau}}} \quad (8)$$

where e is the natural logarithm and τ is the temperature coefficient.

The above traditional Q learning often uses a two-dimensional table to represent the Q function, that is, it is assumed that the states and actions are discrete values. However, for the adaptive bit rate problem, the states, such as bandwidth and buffer, are continuous values, so the states need to be discretized. After discretization of states, some states might not belong to the state set after discretization. Once the agent is in a state that does not belong to the discrete state set, it is difficult to calculate the Q function value of the state, which may reduce the test performance of the algorithm.

Therefore, based on the KNN algorithm[15], we propose an KNN-Q learning algorithm to update the Q function when the state does not belong to the elements in the discrete state set. When the state s_t belongs to the elements in the discrete state set, the Q function is still updated according to (7). However, when the state s_t does not belong to the elements in the discrete state set, K states that are closest to the state s_t are selected from the state set S , and the Q functions corresponding to these K states are updated as follows:

$$Q(knn_t, a_t) = Q(knn_t, a_t) + \eta[r_{t+1} + \gamma \max_b Q(s_{t+1}, b) - \sum_{k=1}^K w_k Q(knn_t, b)] \quad (9)$$

knn_t represents K neighborhood nearest states with the state s_t that does not belong to the elements in the discrete state set S . w_k is coefficient to measure the similarity between the state s_t and the state $s_k \in knn_t$, and w_k is obtained as

$$w_k = \frac{1 / d_k}{\sum_{k=1}^K d_k} \quad (10)$$

where d_k is the distance matrix between the state s_t and the state s_k . s_{t+1} is the next state when agent take action a_t at the state s_t . when $s_{t+1} \notin S$, the Q functions is updated as follows:

$$Q(knn_t, a_t) = Q(knn_t, a_t) + \eta[r_{t+1} + \gamma \max_b \sum_{k=1}^K w'_k Q(knn_{t+1}, b) - \sum_{k=1}^K w_k Q(knn_t, b)], \quad (11)$$

where $Q(knn_{t+1}, b)$ denotes K neighborhood nearest states with the state s_{t+1} . In Algorithm 1, we provide the pseudocode for the proposed KNN-Q learning algorithm.

Algorithm 1 Knn-q training algorithm

1. **Initialize:** SET $Q(S, A)$ arbitrarily;
 2. **REPEAT**(for each episode)
 3. s_t = Observe current state;
 4. **REPEAT**(for each step of episode)
 5. Choose Q value array $Q(s_t, \cdot)$ from $Q(S, A)$
 6. Get a bitrate according to (8), and download a video segment;
 7. Update Buffer $buffer_t$ with (3);
 8. Calculate the Reward R_t with (4);
 9. **IF** $s_t \in S$
 10. Update $Q(S, A)$ with (7);
 11. **ELSE**
 12. Update $Q(S, A)$ with (11);
 13. **END**
 14. s_{t+1} = Observe next state;
 15. $s_t = s_{t+1}$;
 16. **UNTIL** reach the end of an episode
 17. **UNTIL** run out the episodes
-

IV. RESULTS

We carry out a series of simulations to evaluate the performance of the KNN-Q learning algorithm proposed in this paper. We simulate the process of a DASH client requesting an HTTP server to download and play a video using MATLAB. The simulations will set three scenes: "simple", "normal" and "complex" to compare the performance between the KNN-Q learning algorithm and the Q-learning algorithm. The performance of algorithm is evaluated in terms of SSIM. In addition, the influence of parameters on the performance of KNN-Q learning is analyzed.

A. Video Used in Simulations

The videos used in our simulations are downloaded from EvalVid CIF video library (<https://media.xiph.org/video/derf/>) [16]. The relationship between the ρ_a and SSIM of each video in the video library are shown in Fig. 2. In this paper, five videos that marked with thicker lines in Fig. 2 are selected for the simulations. The five videos are named as Brutta, News, Bridge (far), Harbour and Husky. The vector $\mathbf{d}$ for the five videos are show in Table I, and the values of the parameters of each scene in the simulations are shown in the Table II.

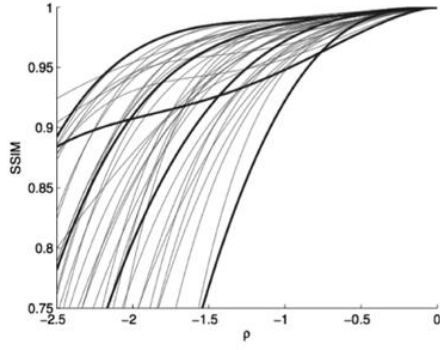


Figure 2. The rate-distortion curves for the videos in the EvalVid CIF video library.

TABLE I. COMPLEXITY COEFFICIENT VECTOR FOR FIVE VIDEOS

Video name	[d1, d2, d3, d4]
Brutta	[-0.0101529, -0.0288832, -0.0242726, 0.0041539]
News	[-0.0106444, -0.0229079, -0.0253096, 0.0007417]
Bridge (far)	[-0.01050829, -0.0538481, -0.0821086, 0.0136133]
Harbour	[-0.0050534, 0.0055396, -0.01726018, 0.0002203]
Husky	[0.0099785, 0.0759046, -0.0113807, 0.0003986]

TABLE II. PARAMETERS IN SIMULATION EXPERIMENTS

Parameter name	Value
η	0.3
γ	0.95
ε	0.3
α	50

β	0.001
B_m	10 sec
Train episodes	50
Test episodes	150
Steps of each episode	800

B. Simulation Results

The simple scene simulates a situation where the network bandwidth is relatively stable, but the video content is relatively simple. Therefore, in the simulation experiment, only “News” video is used in this scene to generate a test video with a single complexity. The range of network bandwidth changes in the scene is [5000,6000] kb/s. The normal scene is the same as the simple scene in that the network bandwidth is relatively stable, but the normal scene uses the five videos. These 5 videos are randomly combined to generate a test video. The size of the network bandwidth in the complex scene varies greatly, and the value range is [400,12500] in kb / s. In the complex scene, all five videos also be used to form a test video.

The comparison between the buffer and video quality of the DASH client in the three scenes is shown in Fig. 3 and Fig. 4. From these figures, we can find that KNN-Q learning and Q-learning algorithms have basically the same buffer growth trend, but KNN-Q learning algorithms require smaller buffer size than Q-learning algorithms in all three scenes (Fig. 3). In addition, the SSIM obtained by the KNN-Q learning algorithm is significantly larger than the SSIM obtained by the Q learning algorithm in all three scenes (Fig. 4).

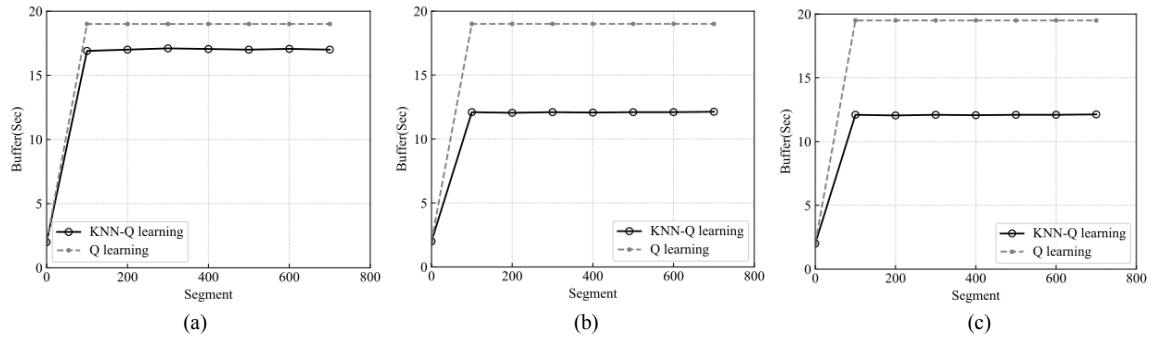


Figure 3. Buffer trend in simple (a), normal (b) and complex (c) scenes.

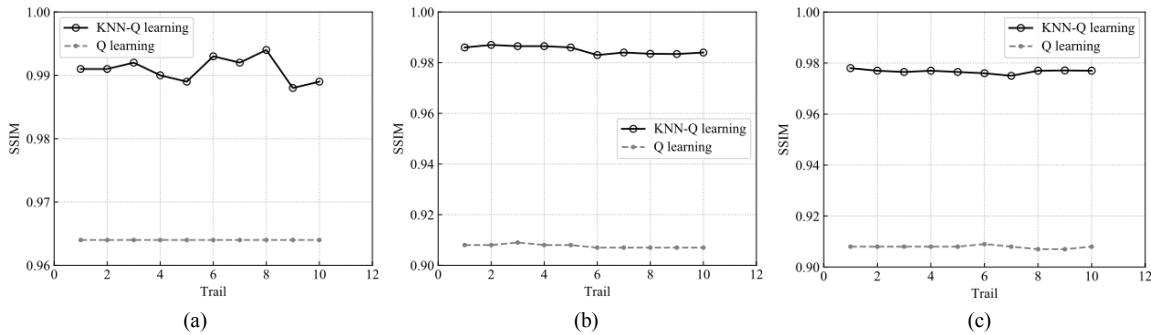


Figure 4. Video quality in simple (a), normal (b) and complex (c) scenes.

We also investigated the effect of different parameter values on the performance of the algorithm. First, when the distance between states is measured by Euler distance, we examine the effect of different K values on the performance of the algorithm. The results indicate the larger the K value, the larger the buffer (Fig. 6) and the smaller the average value of the SSIM (Fig. 5). Especially when $K=6$, the average SSIM value drops rapidly to below 0.91. When K is set to 2, the buffer is at about 13 seconds and its SSIM indicates that its video quality is the best than other values of K .

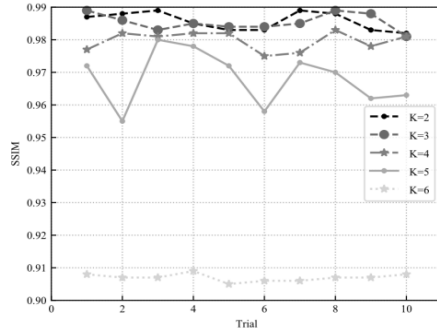


Figure 5. Parameter K on video quality

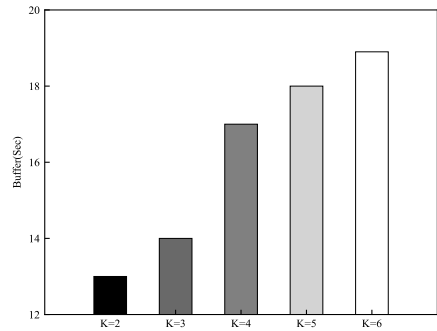


Figure 6. Parameter K on the buffer

V. CONCLUSION

In this work we propose and design a KNN-Q learning DASH client agent. The goal of the agent is to maximize the QoE by adjusting the streaming behavior. The client agent has been provided with a tunable reward function, considering multiple aspects of QoE. The simulation results show that the KNN-Q learning bitrate adaptive algorithm proposed in this paper can obtain higher video quality than the Q-learning rate algorithm. The KNN-Q learning can learn an optimal policy while the states are continuous. For future work, we will verify the algorithm in more realistic scenarios, such as using real TCP packets. In addition, the KNN-Q learning algorithms use Euler distances to measure differences between states. In the future, other more efficient distance measures of states can be considered.

ACKNOWLEDGMENT

This work is supported by the National Natural Science Foundation of China (Grant No. 61976193).

REFERENCES

- [1] Index C.V.N.J.w.p., 'Global mobile data traffic forecast update, 2016–2021', 2017
- [2] I.J.I.m. Sodagar, 'The mpeg-dash standard for multimedia streaming over the internet', *IEEE MultiMedia*, vol.18, April. 2011, pp. 62–67, doi: 10.1109/MMUL.2011.71.
- [3] Y. Xu, Y.Zhou, and D.-M.J.I.T.o.M.C. Chiu, 'Analytical QoE models for bit-rate switching in dynamic adaptive streaming systems', *IEEE Transactions on Mobile Computing*, vol. 13, Dec. 2014, pp. 2734–2748, doi: 10.1109/TMC.2014.2307323.
- [4] A. Bentaleb, B. Taani, A.C. Begen, C. Timmerer, R.J.I.C.S. Zimmermann, and Tutorials: 'A survey on bitrate adaptation schemes for streaming media over HTTP', *IEEE Communications Surveys & Tutorials*, vol. 21, Aug. 2018, pp. 562–585, doi: 10.1109/COMST.2018.2862938.
- [5] M. Claeys, S. Latré, J. Famaey, T. Wu, W. Van Leekwijck, and F. De Turck, "Design of a Q-learning-based client quality selection algorithm for HTTP adaptive video streaming," in *Adaptive and Learning Agents Workshop, part of AAMAS2013, Proceedings, Saint-Paul, USA, 2013*, pp. 30–37.
- [6] S. Akhshabi, A.C. Begen, and C. Dovrolis, 'An experimental evaluation of rate-adaptation algorithms in adaptive streaming over HTTP', *Proc. ACM Conf. on Multimedia Systems*, Jan. 2011, pp. 157–168, doi: 10.1145/1943552.1943574.
- [7] K. Evensen, D. Kaspar, C. Griwodz, P. Halvorsen, A. Hansen, and P. Engelstad, 'Improving the performance of quality-adaptive video streaming over multiple heterogeneous access networks', *Proc. ACM Conf. on Multimedia Systems*, Jan. 2011, pp. 57–68, doi: 10.1145/1943552.1943560.
- [8] M. Claeys, S. Latré, J. Famaey, and F.J.I.C.I. De Turck, 'Design and evaluation of a self-learning HTTP adaptive video streaming client', *IEEE Communications Letters*, vol. 18, Jun. 2014, pp. 716–719, doi: 10.1109/LCOMM.2014.020414.132649.
- [9] M. Claeys, S. Latré, J. Famaey, T. Wu, W. Van Leekwijck, and F.J.C.S. De Turck, 'Design and optimisation of a (FA) Q-learning-based HTTP adaptive streaming client', *Connection Science*, Mar. 2014, vol. 26, pp. 25–43, doi: 10.1080/09540091.2014.885273.
- [10] Zhu H, Cao Y, Wang W, et al. QoE-aware resource allocation for adaptive device-to-device video streaming. *Network IEEE*, 2015, 29(6):6–12, doi: 10.1109/MNET.2015.7340418
- [11] Gadaleta M, Chiariotti F, Rossi M, et al. D-DASH: A Deep Q-Learning Framework for DASH Video Streaming. *IEEE Transactions on Cognitive Communications and Networking*, 2017, 3(4):703–718, doi: 10.1109/TCCN.2017.2755007
- [12] Qiao, G., Leng, S., Maharjan, S., Zhang, Y., and Ansari, N.J.I.I.O.T.J.: 'Deep reinforcement learning for cooperative content caching in vehicular edge computing and networks', *IEEE Internet of Things Journal*, vol. 7, Jan. 2020, pp. 247–257, doi: 10.1109/JIOT.2019.2945640.
- [13] M. Zanforlin, D. Munaretto, A. Zanella, and M. Zorzi, 'SSIM-based video admission control and resource allocation algorithms', 2014 12th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt), May 2014, pp. 656–661, doi: 10.1109/WIOPT.2014.6850361.
- [14] C.J. Watkins, and P.J.M.I. Dayan, 'Q-learning', *Machine Learning*, 1992, vol. 8, pp. 279–292, doi: 10.1007/BF00992698.
- [15] J. de Lope, and D. Maravall, 'The knn-td reinforcement learning algorithm', Third International Work-Conference on the Interplay Between Natural and Artificial Computation, IWINAC 2009, Jun. 2009, pp. 305–314, doi: 10.1007/978-3-642-02264-7_32.
- [16] F. Chiariotti, S. D'Aronco, L. Toni, and P. Frossard, 'Online learning adaptation strategy for DASH clients', *Proceedings of the 7th International Conference on Multimedia Systems, MMSys '16*, May. 2016, pp. 1–12, doi: 10.1145/2910017.2910603.